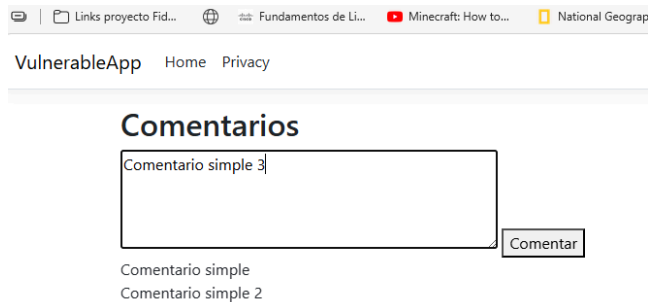
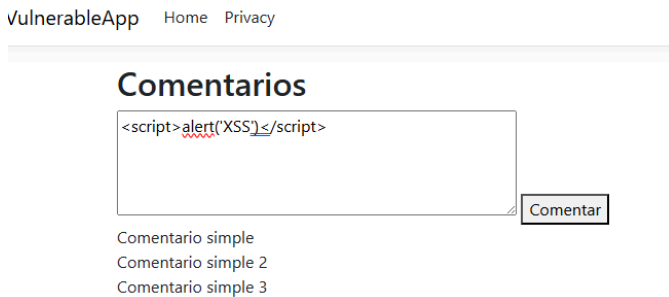


Cross-Site Scripting XSS

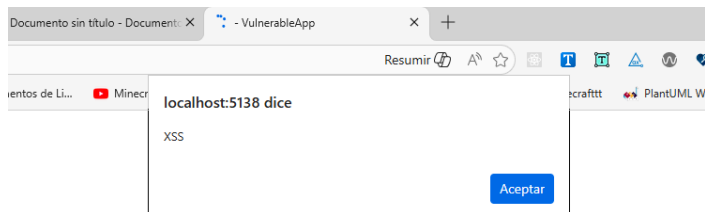
Captura de comentario legítimo:



Captura de contenido no confiable ingresado:



Captura del comportamiento en navegador:



Código vulnerable:

Se encuentra exactamente en Views/Comment/Index.cshtml

```
<div class="comment">@Html.Raw(comment)</div>
```

Si se utilizara el comentario con `@comment`, el framework aplicaría una protección automática llamada Output Encoding. Esta protección convierte caracteres especiales como `<` y `>` en texto seguro para que el navegador no los interprete como código. Así, si un usuario intentara insertar un script, este se mostraría como texto normal en lugar de ejecutarse.

Explicar por qué el navegador interpreta el contenido

Los navegadores no pueden distinguir si el código que reciben proviene del desarrollador o de un usuario malintencionado. Simplemente interpretan y ejecutan el contenido que el

servidor les envía. Cuando un usuario introduce un valor como ese, se guarda sin validación, el contenido se inserta en la página y al cargar la vista interpreta ese texto como un código legítimo ejecutando el JavaScript contenido en ella.

Identificar qué información podría estar en riesgo

Puede robar las cookies para robar el inicio sesión de la cuenta, cambiar cómo se ve una página para engañar al usuario para que ingrese sus datos en formularios falsos o llevarlo a un sitio web malicioso

Relacionar el hallazgo con validación de entrada y codificación de salida

En la validación de entrada la única verificación del ComentController es que el texto no esté vacío, no revisa si tiene etiquetas HTML o scripts. En la codificación de salida la vista evade la protección que tiene ya por defecto el framework.

Medidas correctivas

Codificación de salida:

En la vista dejamos el framework por defecto, ahora el navegador lo verá como simple texto

```
10 <div class="comment">@comment</div>
```

Validaciones en el controlador:

```
if (!string.IsNullOrEmpty(comment))
{
    // convierte cualquier etiqueta script en texto
    // inofensivo "converte": Misspelled word.
    string safeComment = HtmlEncoder.Default.Encode(
        comment);

    _comments.Add(safeComment);
}

return RedirectToAction("Index");
```

Prueba de la medida correctiva implementada:

VulnerableApp Home Privacy

Comentarios

<script>alert('XSS')</script>

Comentar

Comentario simple 1
Comentario simple 2

VulnerableApp Home Privacy

Comentarios

Comentar

Comentario simple 1
Comentario simple 2
Comentario simple 3

<script>alert('XSS')</script>